

ALFA Language Definition

The semantics of ALFA is given in Fig. 0.1 at the end of this document. The syntax are defined below.

Judgement Syntax

In Hazel, proof trees consist of node editors, where each node represents a **Judgement**. A **Judgement** states a logical statement, which serves as the base data for verification. The various **Judgements** supported in Hazel are summarized in Fig. following.

Written Form	Pronunciation	Hazel Deriver Form
$e \Downarrow v$	e evaluates to v	<code>e \=/ v</code>
$e \text{ val}$	e is a value	<code>val e end</code>
$\Gamma \vdash e : \tau$	e has type τ assuming Γ	<code>gamma - e : tau</code>
$\Gamma \vdash e \Rightarrow \tau$	e synthesizes type τ assuming Γ	<code>gamma - e => tau</code>
$\Gamma \vdash e \Leftarrow \tau$	e analyzes against τ assuming Γ	<code>gamma - e <= tau</code>

Context Syntax

A **Context** is anything that appears on the left-hand side of $\vdash$, representing the assumptions under which a **Judgement** holds. Contexts in Hazel is essentially a list of **Proposition**, and they can be manipulated using various operations. The key context operations/notations in Hazel are as follows:

- Empty Context: `[]`
- Context Literal: `[p1, p2, ...]`
- Context Append: `gamma_1 @ gamma_2`
- Context Cons: `p :: gamma`

External Variable Binding Syntax

Except making subtree abbreviation (as clearly instructed in the UI). You can also bind proof tree syntax to Hazel variable and later use in proof trees. This allows you to reuse variables and simplify complex derivations.

- Judgement: `of_jdmt ... end`
- Context: `of_ctx ... end`
- Proposition: `of_prop ... end`
- ALFA Expression: `of_alfa_exp ... end`
- ALFA Type: `of_alfa_typ ... end`

		Structural Form	Concrete Form	Hazel Deriver Form
Typ τ	$::=$	Num	Num	Num
		Bool	Bool	Bool
		Arrow($\tau_{\text{in}}, \tau_{\text{out}}$)	$\tau_{\text{in}} \rightarrow \tau_{\text{out}}$	tau_in -> tau_out
		Prod(τ_1, τ_2)	$\tau_1 \times \tau_2$	tau1 * tau2
		Unit	Unit	Unit
		Sum(τ_L, τ_R)	$\tau_L + \tau_R$	tauL + tauR
Expr e, v	$::=$	NumLit[n]	$\underline{n}$	n (Number Literal)
		UnOp[u](e)	ue	u(e)
		BinOp[b](e_1, e_2)	$e_1 b e_2$	e1 b e2
		True	True	True
		False	False	False
		If(e_1, e_2, e_3)	if e_1 then e_2 else e_3	if e1 then e2 else e3
		Var[x]	x	x
		LetAnn($\tau?, e_1, x.e_2$)	let $x : \tau$ be e_1 in e_2	let x : tau = e1 in e2
			let x be e_1 in e_2	let x = e1 in e2
		Fun($\tau_{\text{in}}?, x.e$)	fun ($x : \tau_{\text{in}}$) $\rightarrow e$	fun x : tau_in -> e
			fun $x \rightarrow e$	fun x -> e
		Ap(e_1, e_2)	$e_1 e_2$	e1(e2)
		Pair(e_1, e_2)	(e_1, e_2)	(e1, e2)
		Triv	()	()
		PrjL(e)	$e.\text{fst}$	e.fst
		PrjR(e)	$e.\text{snd}$	e.snd
		LetPair($e_1, x.y.e_2$)	let (x, y) be e_1 in e_2	let (x, y) = e1 in e2
		InjL(e)	L e	L(e)
		InjR(e)	R e	R(e)
		Case($e_1, x.e_2, y.e_3$)	case e_1 of L(x) $\rightarrow e_2$ else R(y) $\rightarrow e_3$	case e1 L(x) => e2 R(y) => e3 end
UnOp u	$::=$	OpNeg	—	—
BinOp b	$::=$	OpLt	<	<
		OpGt	>	>
		OpEq	=?	==
		OpPlus	+	+
		OpMinus	—	—
		OpTimes	*	*

Figure 0.1: Syntax of ALFA with Hazel Forms

0.1 Type System

The declarative type system for ALFA is defined by the typing judgement (Sec. 0.1).

Typing

A derivation of the typing judgement, $\Gamma \vdash e : \tau$ establishes that τ is a type for e assuming a typing context Γ of the form:

$$x_1 : \tau_1, \dots, x_n : \tau_n \quad (n \geq 0)$$

The typing judgement is defined by the following rules.

Booleans

$$\begin{array}{c} \frac{}{\Gamma \vdash \text{True} : \text{Bool}} \quad (\text{T-True}) \qquad \frac{}{\Gamma \vdash \text{False} : \text{Bool}} \quad (\text{T-False}) \\[10pt] \frac{\Gamma \vdash e_1 : \text{Bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \tau} \quad (\text{T-If}) \end{array}$$

Numbers

$$\begin{array}{c} \frac{}{\Gamma \vdash \underline{n} : \text{Num}} \quad (\text{T-Num}) \qquad \frac{\Gamma \vdash e : \text{Num}}{\Gamma \vdash -e : \text{Num}} \quad (\text{T-Neg}) \\[10pt] \frac{\Gamma \vdash e_1 : \text{Num} \quad \Gamma \vdash e_2 : \text{Num}}{\Gamma \vdash e_1 + e_2 : \text{Num}} \quad (\text{T-Plus}) \qquad \frac{\Gamma \vdash e_1 : \text{Num} \quad \Gamma \vdash e_2 : \text{Num}}{\Gamma \vdash e_1 - e_2 : \text{Num}} \quad (\text{T-Minus}) \\[10pt] \frac{\Gamma \vdash e_1 : \text{Num} \quad \Gamma \vdash e_2 : \text{Num}}{\Gamma \vdash e_1 * e_2 : \text{Num}} \quad (\text{T-Times}) \qquad \frac{\Gamma \vdash e_1 : \text{Num} \quad \Gamma \vdash e_2 : \text{Num}}{\Gamma \vdash e_1 < e_2 : \text{Bool}} \quad (\text{T-Lt}) \\[10pt] \frac{\Gamma \vdash e_1 : \text{Num} \quad \Gamma \vdash e_2 : \text{Num}}{\Gamma \vdash e_1 > e_2 : \text{Bool}} \quad (\text{T-Gt}) \qquad \frac{\Gamma \vdash e_1 : \text{Num} \quad \Gamma \vdash e_2 : \text{Num}}{\Gamma \vdash e_1 =? e_2 : \text{Bool}} \quad (\text{T-Eq}) \end{array}$$

Variables

$$\begin{array}{c} \frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau} \quad (\text{T-Var}) \qquad \frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \text{let } x : \tau_1 \text{ be } e_1 \text{ in } e_2 : \tau_2} \quad (\text{T-LetAnn}) \\[10pt] \frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \text{let } x \text{ be } e_1 \text{ in } e_2 : \tau_2} \quad (\text{T-Let}) \end{array}$$

Functions

$$\begin{array}{c}
\frac{\Gamma, x : \tau_{\text{in}} \vdash e : \tau_{\text{out}}}{\Gamma \vdash \text{fun } x \rightarrow e : \tau_{\text{in}} \rightarrow \tau_{\text{out}}} \quad (\text{T-Fun}) \qquad \frac{\Gamma, x : \tau_{\text{in}} \vdash e : \tau_{\text{out}}}{\Gamma \vdash \text{fun } (x : \tau_{\text{in}}) \rightarrow e : \tau_{\text{in}} \rightarrow \tau_{\text{out}}} \quad (\text{T-FunAnn}) \\
\\
\frac{\Gamma \vdash e_1 : \tau_{\text{in}} \rightarrow \tau_{\text{out}} \quad \Gamma \vdash e_2 : \tau_{\text{in}}}{\Gamma \vdash e_1 \ e_2 : \tau_{\text{out}}} \quad (\text{T-Ap})
\end{array}$$

Products

$$\begin{array}{c}
\frac{}{\Gamma \vdash () : \text{Unit}} \quad (\text{T-Triv}) \qquad \frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash (e_1, e_2) : \tau_1 \times \tau_2} \quad (\text{T-Pair}) \\
\\
\frac{\Gamma \vdash e_1 : \tau_1 \times \tau_2 \quad \Gamma, x : \tau_1, y : \tau_2 \vdash e_2 : \tau}{\Gamma \vdash \text{let } (x, y) \text{ be } e_1 \text{ in } e_2 : \tau} \quad (\text{T-LetPair}) \qquad \frac{\Gamma \vdash e : \tau_1 \times \tau_2}{\Gamma \vdash e.\text{fst} : \tau_1} \quad (\text{T-PrjL}) \\
\\
\frac{\Gamma \vdash e : \tau_1 \times \tau_2}{\Gamma \vdash e.\text{snd} : \tau_2} \quad (\text{T-PrjR})
\end{array}$$

Sums

$$\begin{array}{c}
\frac{\Gamma \vdash e : \tau_1}{\Gamma \vdash \text{L } e : \tau_1 + \tau_2} \quad (\text{T-InjL}) \qquad \frac{\Gamma \vdash e : \tau_2}{\Gamma \vdash \text{R } e : \tau_1 + \tau_2} \quad (\text{T-InjR}) \\
\\
\frac{\Gamma \vdash e : \tau_1 + \tau_2 \quad \Gamma, x : \tau_1 \vdash e_1 : \tau \quad \Gamma, y : \tau_2 \vdash e_2 : \tau}{\Gamma \vdash \text{case } e \text{ of } \text{L}(x) \rightarrow e_1 \text{ else } \text{R}(y) \rightarrow e_2 : \tau} \quad (\text{T-Case})
\end{array}$$

0.2 Evaluation Semantics

The evaluation semantics of Recursive ALFA is defined by two judgements: the value judgement (Sec. 0.2) and the evaluation judgement (Sec. 0.2).

Values

A derivation of the value judgement, $e \text{ val}$, establishes that the Recursive ALFA expression e is a value.

$$\begin{array}{c}
\frac{}{\underline{n} \text{ val}} \text{ (V-Num)} \qquad \frac{}{\text{True val}} \text{ (V-True)} \qquad \frac{}{\text{False val}} \text{ (V-False)} \\
\\
\frac{}{\text{fun } (x : \tau) \rightarrow e \text{ val}} \text{ (V-Fun)} \qquad \frac{e_1 \text{ val} \quad e_2 \text{ val}}{(e_1, e_2) \text{ val}} \text{ (V-Pair)} \qquad \frac{}{() \text{ val}} \text{ (V-Triv)} \\
\\
\frac{e \text{ val}}{\text{L } e \text{ val}} \text{ (V-InjL)} \qquad \frac{e \text{ val}}{\text{R } e \text{ val}} \text{ (V-InjR)}
\end{array}$$

Evaluation

A derivation of the evaluation judgement, $e \Downarrow e'$, establishes that e' is the value of e .

Value Evaluation Values evaluate to themselves according to the following rule.

$$\frac{e \text{ val}}{e \Downarrow e} \text{ (E-Val)}$$

Booleans

$$\begin{array}{c}
\frac{e_1 \Downarrow \text{True} \quad e_2 \Downarrow e'_2}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Downarrow e'_2} \text{ (E-If-T)} \qquad \frac{e_1 \Downarrow \text{False} \quad e_3 \Downarrow e'_3}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Downarrow e'_3} \text{ (E-If-F)}
\end{array}$$

Numbers

$$\begin{array}{c}
\frac{e \Downarrow \underline{n}}{-e \Downarrow \underline{-n}} \text{ (E-Neg)} \qquad \frac{e_1 \Downarrow \underline{n_1} \quad e_2 \Downarrow \underline{n_2}}{e_1 + e_2 \Downarrow \underline{n_1 + n_2}} \text{ (E-Plus)} \qquad \frac{e_1 \Downarrow \underline{n_1} \quad e_2 \Downarrow \underline{n_2}}{e_1 - e_2 \Downarrow \underline{n_1 - n_2}} \text{ (E-Minus)} \\
\\
\frac{e_1 \Downarrow \underline{n_1} \quad e_2 \Downarrow \underline{n_2}}{e_1 * e_2 \Downarrow \underline{n_1 * n_2}} \text{ (E-Times)} \qquad \frac{e_1 \Downarrow \underline{n_1} \quad e_2 \Downarrow \underline{n_2} \quad n_1 < n_2}{e_1 < e_2 \Downarrow \text{True}} \text{ (E-Lt-T)} \\
\\
\frac{e_1 \Downarrow \underline{n_1} \quad e_2 \Downarrow \underline{n_2} \quad n_1 \geq n_2}{e_1 < e_2 \Downarrow \text{False}} \text{ (E-Lt-F)} \qquad \frac{e_1 \Downarrow \underline{n_1} \quad e_2 \Downarrow \underline{n_2} \quad n_1 > n_2}{e_1 > e_2 \Downarrow \text{True}} \text{ (E-Gt-T)} \\
\\
\frac{e_1 \Downarrow \underline{n_1} \quad e_2 \Downarrow \underline{n_2} \quad n_1 \leq n_2}{e_1 > e_2 \Downarrow \text{False}} \text{ (E-Gt-F)} \qquad \frac{e_1 \Downarrow \underline{n} \quad e_2 \Downarrow \underline{n}}{e_1 =? e_2 \Downarrow \text{True}} \text{ (E-Eq-T)} \\
\\
\frac{e_1 \Downarrow \underline{n_1} \quad e_2 \Downarrow \underline{n_2} \quad n_1 \neq n_2}{e_1 =? e_2 \Downarrow \text{False}} \text{ (E-Eq-F)}
\end{array}$$

Let Expressions

$$\frac{e_1 \Downarrow e'_1 \quad [e'_1/x]e_2 \Downarrow e'_2}{\text{let } x : \tau? \text{ be } e_1 \text{ in } e_2 \Downarrow e'_2} \quad (\text{E-Let})$$

Function Application

$$\frac{e_1 \Downarrow \text{fun } (x : \tau?) \rightarrow e \quad e_2 \Downarrow e'_2 \quad [e'_2/x]e \Downarrow e'}{e_1(e_2) \Downarrow e'} \quad (\text{E-Ap})$$

Products

$$\frac{e_1 \Downarrow e'_1 \quad e_2 \Downarrow e'_2}{(e_1, e_2) \Downarrow (e'_1, e'_2)} \quad (\text{E-Pair}) \quad \frac{e \Downarrow (e_1, e_2)}{e.\text{fst} \Downarrow e_1} \quad (\text{E-PrjL}) \quad \frac{e \Downarrow (e_1, e_2)}{e.\text{snd} \Downarrow e_2} \quad (\text{E-PrjR})$$

$$\frac{e_1 \Downarrow (e_{11}, e_{12}) \quad [e_{11}/x_1][e_{12}/x_2]e_2 \Downarrow e_3}{\text{let } (x_1, x_2) \text{ be } e_1 \text{ in } e_2 \Downarrow e_3} \quad (\text{E-LetPair})$$

Sums

$$\frac{e \Downarrow e'}{\text{L } e \Downarrow \text{L } e'} \quad (\text{E-InjL}) \quad \frac{e \Downarrow e'}{\text{R } e \Downarrow \text{R } e'} \quad (\text{E-InjR})$$

$$\frac{e \Downarrow \text{L } e_L \quad [e_L/x_1]e_1 \Downarrow e'_1}{\text{case } e \text{ of } \text{L}(x_1) \rightarrow e_1 \text{ else } \text{R}(x_2) \rightarrow e_2 \Downarrow e'_1} \quad (\text{E-CaseL})$$

$$\frac{e \Downarrow \text{R } e_R \quad [e_R/x_2]e_2 \Downarrow e''_2}{\text{case } e \text{ of } \text{L}(x_1) \rightarrow e_1 \text{ else } \text{R}(x_2) \rightarrow e_2 \Downarrow e'_2} \quad (\text{E-CaseR})$$